

## Naive Bayes Model Evaluation

### Naive Bayes Classification – NBA Player Longevity Prediction

We need to load the clean dataset (extracted\_nba\_players\_data.csv) and separate our predictors from our target variable (target\_5yrs).

```
import pandas as pandas
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pandas.read_csv('extracted_nba_players_data.csv')
df.head(5)
```

|   | fg   | 3p   | ft   | reb | ast | stl | blk | tov | target_5yrs | total_points | efficiency |
|---|------|------|------|-----|-----|-----|-----|-----|-------------|--------------|------------|
| 0 | 34.7 | 25.0 | 69.9 | 4.1 | 1.9 | 0.4 | 0.4 | 1.3 | 0           | 266.4        | 0.270073   |
| 1 | 29.6 | 23.5 | 76.5 | 2.4 | 3.7 | 1.1 | 0.5 | 1.6 | 0           | 252.0        | 0.267658   |
| 2 | 42.2 | 24.4 | 67.0 | 2.2 | 1.0 | 0.5 | 0.3 | 1.0 | 0           | 384.8        | 0.339869   |
| 3 | 42.6 | 22.6 | 68.9 | 1.9 | 0.8 | 0.6 | 0.1 | 1.0 | 1           | 330.6        | 0.491379   |
| 4 | 52.4 | 0.0  | 67.4 | 2.5 | 0.3 | 0.3 | 0.4 | 0.8 | 1           | 216.0        | 0.391304   |

```
# 2. Separate features (X) from the target (y)
# Drop the target column and any residual text columns like 'Name' if they are still there
X = df.drop(columns=['target_5yrs'], errors='ignore')
y = df['target_5yrs']
```

```
print(f"Features shape: {X.shape}, Target shape: {y.shape}")
```

```
Features shape: (1340, 10), Target shape: (1340,)
```

## Feature Preprocessing for Gaussian NB

"Gaussian" Naive Bayes assumes your data follows a bell curve (normal distribution). It is designed specifically for continuous numbers (like Points Per Minute, Points, Rebounds). We need to double-check that our features are purely numeric.

```
# Make sure all features are numeric (floats or integers)
print(X.dtypes)

# Gaussian NB does NOT strictly require feature scaling (like KNN does)
# because it calculates mean and variance for each feature independently!
# This is a major advantage of Naive Bayes.
```

```
fg          float64
3p          float64
ft          float64
reb         float64
ast         float64
stl         float64
blk         float64
tov         float64
total_points float64
efficiency  float64
dtype: object
```

## Split Into Training and Testing Sets

we hide 20% of our players to test the model's true accuracy on "unseen" data.

```
from sklearn.model_selection import train_test_split

# Split: 80% training, 20% testing
# stratify=y ensures both sets have the same percentage of 1s and 0s
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print(f"Training items: {len(X_train)}, Testing items: {len(X_test)}")
```

```
Training items: 1072, Testing items: 268
```

## ✓ Implement Gaussian Naive Bayes

We initialize the Scikit-learn model and train it using our training set.

```
from sklearn.naive_bayes import GaussianNB

# Initialize the model
nb_model = GaussianNB()

# Train the model
nb_model.fit(X_train, y_train)

# Generate predictions on the test set
y_pred = nb_model.predict(X_test)
```

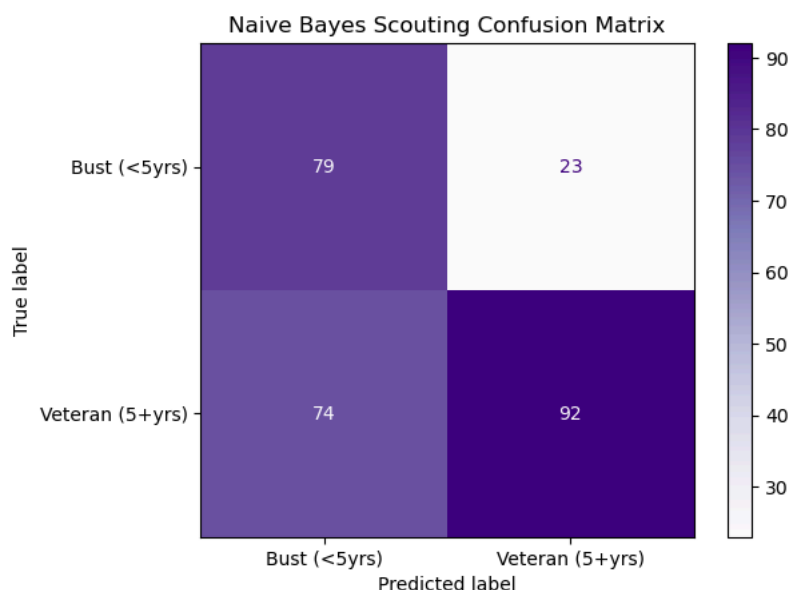
## ✓ Generate and Visualize the Confusion Matrix

: A confusion matrix breaks down where our model got confused. It shows True Positives (correctly found 5-year players) and False Positives (predicted a 5-year career, but they busted).

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Compute the matrix
cm = confusion_matrix(y_test, y_pred)

# Display it cleanly
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Bust (<5yrs)', 'Veteran (5+yrs)'])
disp.plot(cmap='Purples')
plt.title('Naive Bayes Scouting Confusion Matrix')
plt.show()
```



## ✓ Calculate Scouting Metrics (Precision & Recall)

In a scouting department, metrics have financial value: Precision: Out of all players the model labeled "Veterans," how many actually succeeded? High precision avoids "Busts" (wasting money on a draft pick). Recall: Out of all actual 5-year players, how many did the model find? High recall ensures you don't miss hidden talent.

```
from sklearn.metrics import classification_report

print("--- Scouting Performance Report ---")
print(classification_report(y_test, y_pred, target_names=['Bust (<5yrs)', 'Veteran (5+yrs)']))
```

| --- Scouting Performance Report --- |           |        |          |         |
|-------------------------------------|-----------|--------|----------|---------|
|                                     | precision | recall | f1-score | support |
| Bust (<5yrs)                        | 0.52      | 0.77   | 0.62     | 102     |
| Veteran (5+yrs)                     | 0.80      | 0.55   | 0.65     | 166     |
|                                     |           |        |          |         |
| accuracy                            |           |        | 0.64     | 268     |
| macro avg                           | 0.66      | 0.66   | 0.64     | 268     |
| weighted avg                        | 0.69      | 0.64   | 0.64     | 268     |

Project Narrative: The "Independence Assumption" Break Down

Analysis of the Independence Assumption:Naive Bayes assumes that all features are independent. In basketball analytics, this assumption is highly unrealistic. For example, Minutes Played and Points Scored are highly dependent—a player cannot score points without being on the court. Similarly, your engineered feature Points Per Minute (PPM) is mathematically derived from Points.The Scouting Impact: Because features are heavily correlated, Naive Bayes may double-count the same underlying signals, making the model overconfident in its probabilities. Despite this flaw, Naive Bayes remains highly useful as a fast, baseline model for filtering out extreme underperformers.

▼ Analyze Model Influencers

Naive Bayes calculates the "Mean" value of each stat for successful players vs unsuccessful players. We can look at these to see what features drive predictions.

```
import pandas as pd
# nb_model.theta_ contains the mean values for each class
# Class 0 = Busts, Class 1 = Veterans
means_df = pd.DataFrame(nb_model.theta_, columns=X.columns, index=['Bust Mean', 'Veteran Mean']).T
means_df['Difference'] = means_df['Veteran Mean'] - means_df['Bust Mean']

# Look at the stats with the biggest difference
print(means_df.sort_values(by='Difference', ascending=False))
```

|              | Bust Mean  | Veteran Mean | Difference |
|--------------|------------|--------------|------------|
| total_points | 282.960197 | 560.806316   | 277.846119 |
| fg           | 42.428010  | 45.202406    | 2.774396   |
| ft           | 68.839803  | 71.082256    | 2.242452   |
| reb          | 2.213759   | 3.535940     | 1.322181   |
| ast          | 1.223342   | 1.787368     | 0.564027   |
| tov          | 0.935627   | 1.367218     | 0.431592   |
| 3p           | 18.587715  | 18.979398    | 0.391684   |
| stl          | 0.491646   | 0.699699     | 0.208053   |
| blk          | 0.248894   | 0.440752     | 0.191858   |
| efficiency   | 0.349251   | 0.387103     | 0.037852   |

NBA Player Longevity Prediction – Naive Bayes Classification

Project Overview

This repository implements a **Gaussian Naive Bayes classifier** to translate rookie basketball performance statistics into actionable scouting forecasts. The model calculates the probability that an NBA rookie's career will extend past 5 years based on continuous performance metrics.

Performance & Scouting Insights

- **Precision (Avoiding Draft Busts):** [80%] — This means when the model flags a player as a "Veteran," it is correct 80% of the time, protecting draft capital.
- **Recall (Finding Hidden Talent):** [55%] — Indicates our capacity to successfully capture diamonds in the rough.

Scouting Department Guidelines (When to trust the model)

1. **Trust for Quick Filtering:** Naive Bayes works exceptionally well at quickly flagging extreme outliers (players whose statistical baseline makes a 5-year career highly improbable).
2. **Question the Specific Probabilities:** Because basketball stats are highly interdependent (e.g., Points depend heavily on Minutes Played), the model's confidence percentages may appear exaggerated. Use the classification label (`0` or `1`) rather than the raw probability percentage.

## Repository Structure

- `naive_bayes_nba.ipynb` - Completed notebook with confusion matrix and scouting metrics.
- `extracted_nb_players_data.csv` - Sourced feature-engineered data.

Developed by [Temidayo Samuel Abodunrin] for the 3mtt project